

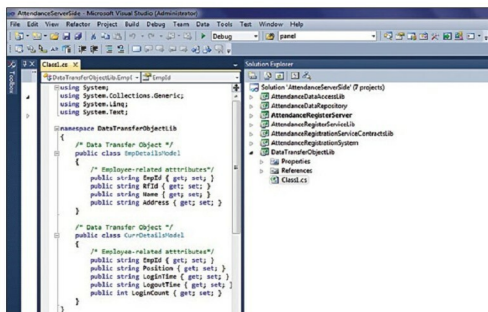


Fig. 13: Public library EmpDetailsModel and CurrDetailsModel

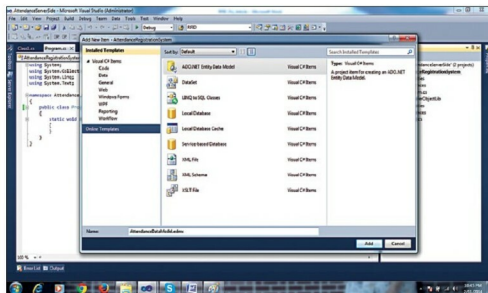


Fig. 14: ADO.NET Entity Data Model

3. Go to Program.cs of AttendanceRegistrationSystem project and make Class Program public. Add a new item of type ADO.NET Entity Data Model to project AttendanceRegistrationSystem from Project/Add New as shown Fig. 14. Name it AttendanceDataModel.

4. From Entity Data Model wizard, select Generate from Database by right-clicking on the screen, and click Next as shown in Fig. 15.

5. A dialog box appears. Choose Data Connection from already existing connections or create a new one. From the drop-down menu, select your created database for this application. For our application we will

select EmployeeAttendanceDB. Now, click Next.

Note. The entity connection string is created for the database and it will be appended to App. Config of our AttendanceRegistrationSystem

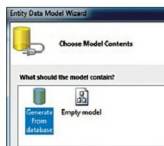


Fig. 15: Generating from the database

application. Save entity connection as EmployeeAttendanceDBEntities, which is also added to App.Config as shown in Fig. 16.

6. Choose tables from the list of database objects. Here, a relational data model of the name EmployeeAttendanceDBModel gets created. Finally, click Finish.

Now, we can see the structure of our AttendanceDataModel.edmx as shown in Fig. 17.

7. To create data-access layer, first create the data repository by following the interface approach. Thus, we create a class library AttendanceDataRepository and add an interface to it by the name IAttendanceDataRepository.

In this project, we also have to add reference of our DataTransferObjectLib, so that we can use DTO model classes further. This can be done by right-clicking on References of AttendanceDataRepository project and then selecting Add Reference. A

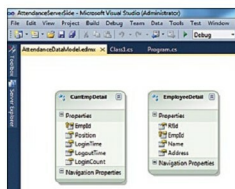


Fig. 17: AttendanceDataModel.edmx

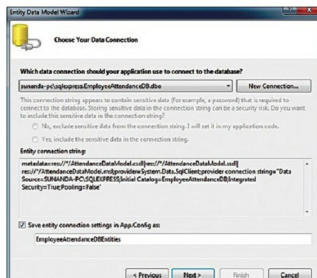


Fig. 16: Saving entity connection